

<통신시스템 캡스톤 디자인1>

3차보고서

-4.7-

주요계획

1. PC 기반 FastAPI 서버 구축 ✓
2. 기본 API(GET/POST) 작성 ✓
3. 라즈베리파이<-> 서버 통신 확인 ✓

목표: 라즈베리파이와 PC 서버 간 통신 가능
상태 확보

-팀원-

오석준

정민영

황다원



체크리스트 및 역할 분담

DAY	주요 계획	목표	비고
3.31	Raspberry Pi OS 기반 환경 점검 SSH 원격 접속 구성 네트워크 및 IP 확인 Python/pip/venv 환경 구성, YOLO 설치 및 실행 가능성 검토	라즈베리파이 개발 환경 구축 완료 YOLO 실행 오류 (Illegal instruction) 확인	완료
4.7	PC 기반 FastAPI 서버 구축 기본 API(GET/POST) 작성 라즈베리파이↔서버 통신 확인	라즈베리파이와 PC 서버 간 통신 가능 상태 확보	완료
4.14	USB 웹캠(Logitech C920) 연결 카메라 인식 확인 OpenCV 기반 영상 캡처 및 이미지 저장	영상 입력 환경 구축 완료	
4.21	라즈베리파이에서 캡처한 이미지를 PC 서버로 전송 HTTP POST 방식 업로드 기능 구현	이미지 전송 기능 구현	
4.28	PC 서버에서 YOLOv8n/v8s 실행 환경 구성 이미지 기반 객체 탐지 테스트	객체 탐지 기능 1차 구현	
5.5	탐지 결과 기반 침입 판단 로직 구현 ROI(제한 영역) 설정	단순 인식에서 침입 판단 기능으로 확장	
5.12	등록 얼굴 데이터 수집, 얼굴 검출 및 특정 추출 방식 설계, 등록 인물과 미등 록 인물을 구분하기 위한 얼굴 인식 구 조 설계, 결과 저장을 위한 DB 구조 설 계	얼굴 인식 엔진 및 데 이터 저장 체계 확립	
5.19	Opencv 기반 얼굴 검출과 특징 비교 기 능 구현, 등록 인물/미등록 인물 판별 테 스트, yolo 사람 탐지 결과와 얼굴 인식 결과를 결합한 판별하는 로직을 구현	등록 인물과 외부인 구 분 기능 구현, 객체 탐 지와 얼굴 인식을 결합 한 판별 구조 확인	
5.26	이벤트 정의 및 알림 기능 구현 침입 발생 시 로그/알림 생성	이벤트 기반 시스템 완성	
6.2	전체 시스템 통합 테스트 오류 수정 및 성능 개선	1학기 1차 완성본 도출	

회차	오석준 처리,YOLO,서버	o/x	정민영 입력,전송,통신	0/x	황다원 DB,UI,데이터	o/x
2차 3.31	1. Raspberry Pi 환경 점검 및 YOLO 실행 가능성 검토 2. Python 개발 환경 구축 및 YOLO 설치 후 실행 테스트 수행 - 세부업무: 기존 OS 상태 확인, 가상 환경 구성, YOLO 설치 및 import 테스트, 실행 오류(Illegal instruction) 발생 원인 분석	o	1. Raspberry Pi 네트워크 및 IP 확인을 통한 통신 환경 점검 2. YOLO 실행 결과 검토 및 보고서 작성, 발표 준비 - 세부업무: SSH 접속 및 IP 확인, 네트워크 상태 점검, 오류 로그 분석, 구조 변경 방향 문서화	o	1. Raspberry Pi 관련 자료 조사 및 OS 설치, SSH 환경 구성 2. 개발 환경 구축 지원 및 프로젝트 준비 사항 정리 - 세부업무: Raspberry Pi OS 설치, SSH 설정, 개발 환경 구성 지원, 물품 조사 및 정리	o
3차 4.7	1. FastAPI 서버 구축 및 API 구현 2. 서버 요청 처리 로직 구성 - 세부업무: 서버 실행 환경 구성, GET/POST API 작성, 요청 처리 흐름 설계, 서버 실행 테스트	o	1. Raspberry Pi → 서버 데이터 전송 구현 2. 통신 테스트 및 연결 검증 - 세부업무: HTTP 요청 코드 작성, 서버 API 호출, 응답 확인, 통신 오류 점검	o	1. 데이터 흐름 구조 설계 2. API 데이터 형식 정의 - 세부업무: 요청/응답 JSON 구조 설계, 전체 시스템 흐름 문서화, 통신 구조 정리	o
4차 4.21	1. YOLOv8 모델을 서버에 적용하여 객체 탐지 기능 구현 2. 이미지 입력 시 사람 객체 탐지 및 결과 반환 로직 개발 - 세부업무: YOLO 모델 로드, 이미지 입력 처리, 사람 객체 필터링, 결과 반환 구현		1. Raspberry Pi 이미지 캡처 및 서버 전송 구조 구현 2. 전송 이미지 해상도 및 용량 조절을 통한 입력 데이터 최적화 - 세부업무: OpenCV 기반 캡처, 이미지 전송 코드 구현, 해상도 조정, 전송 속도 테스트		1. YOLO 탐지 결과 데이터 구조(JSON) 설계 2. 탐지 결과 처리 기준 정의 및 검증 - 세부업무: bounding box 구조 정의, JSON 포맷 설계, 탐지 결과 검증 및 데이터 정합성 확인	
5차 4.28	1. YOLO 결과 처리 로직 구현 2. 탐지 데이터 변환 - 세부업무: bounding box 좌표		1. 이미지 전송 주기 설정 2. 전송 안정성 테스트 - 세부업무: 전송 간		1. DB 저장 구조 설계 2. 데이터 저장 기능 구현 - 세부업무: 테이블	

	추출, detected 값 생성, 데이터 구조 변환		격 조정, 반복 전송 테스트, 서버 응답 확인		설계, insert 로직 구현, 저장 데이터 검증	
6차 5.5	1. 침입 판정 로직 구현 2. ROI 기반 판단 기능 개발 - 세부업무: 중심점 계산, ROI 영역 설정, 침입 조건 정의		1. 테스트 환경 구축 2. 데이터 수집 및 검증 - 세부업무: 카메라 위치 설정, 테스트 영상 확보, 다양한 상황 실험		1. 이벤트 데이터 구조 설계 2. DB 반영 및 저장 기능 구현 - 세부업무: event_type 정의, DB 스키마 수정, 결과 저장 구현	
7차 5.12	1. 이벤트 생성 로직 구현 2. 중복 방지 처리 - 세부업무: 상태 변화 감지, 이벤트 발생 조건 설정, 시간 기반 제어		1. 연속 전송 테스트 2. 시스템 동작 검증 - 세부업무: 반복 전송 테스트, 이벤트 발생 흐름 확인		1. 이벤트 DB 구축 2. 데이터 관리 기능 구현 - 세부업무: events 테이블 생성, 저장 로직 구현, 데이터 확인	
8차 5.19	1. YOLO 모델(v8n, v8s) 성능 비교 및 최적 모델 선정 2. 객체 탐지 정확도 및 처리 속도 분석 - 세부업무: 모델 비교, 처리 시간 측정, 정확도 분석		1. 이미지 전송 속도 및 시스템 FPS 측정 2. 전송 효율 및 네트워크 성능 분석 - 세부업무: FPS 계산, latency 측정, 전송 주기 비교, 전송 구조 최적화 및 네트워크 병목 분석		1. DB 조회 성능 개선 및 인덱스 적용 2. 데이터 증가에 따른 DB 처리 속도 분석 - 세부업무: 인덱스 설정, 조회 속도 비교, 성능 분석	
9차 5.26	1. API 기능 확장 2. 데이터 처리 기능 구현 - 세부업무: 로그 조회 API, 이벤트 조회 API, JSON 응답 처리		1. 데이터 수집 및 테스트 2. 통신 안정성 검증 - 세부업무: 다양한 환경 테스트, 전송 오류 확인		1. UI 구현 2. 데이터 시각화 - 세부업무: 로그 화면 구성, 이미지 출력, UI 구성	
10차 6.2	1. 서버 구조 개선 2. 동시 요청 처리 - 세부업무: 멀티 요청 처리, 서버 안정성 확보		1. 다중 Pi 구성 2. 동시 전송 테스트 - 세부업무: Pi 2대 설정, 동시 요청 테스트		1. 데이터 구분 구조 설계 2. 카메라별 관리 기능 구현 - 세부업무: camera_id 추가, 데이터 분리 저장	
11차 6.xx	1. 서버 안정화 2. 예외 처리 구현 - 세부업무: 오류 처		1. 실시간 테스트 2. 시스템 검증 - 세부업무: 전체 흐		1. UI 개선 2. 사용자 기능 보완 - 세부업무: 정렬, 필	

	리 로직, 로그 관리		름 테스트, 문제 상황 확인		터, 화면 개선	
	1. 시스템 정리 2. 기술 문서화 - 세부업무: 구조 정리, 핵심 기능 정리		1. 시연 준비 2. 테스트 및 영상 제작 - 세부업무: 시연 영상 제작, 테스트 수행		1. 데이터 분석 2. 결과 시각화 - 세부업무: 통계 정리, 그래프 생성	

1. 수행 목표

전체 시스템: Pi -> Server -> YOLO -> Opencv -> DB -> UI

3차(전체 시스템의 연결이 되는지 확인하는 단계): 이미지 -> 서버

3차에서는 클라이언트와 서버 간 통신이 정상적으로 이루어지는지 확인하고, 이미지 업로드 및 저장 기능을 구현하는 것을 목표로 한다.

이를 통해 이후 YOLO 객체 탐지 기능을 적용하기 위한 기본 구조를 구성하였다.

2. 시스템 구성

클라이언트: Raspberry Pi(테스트에서는 노트북 원격 연결)

서버: FastAPI 기반 서버(동일 노트북)

클라이언트는 이미지를 서버로 전송하고, 서버는 해당 이미지를 저장한 후 결과를 반환하도록 구성하였다.

3. 구현 내용

(1) 서버 상태 확인 API 구현

서버가 정상적으로 실행되는지 확인하기 위해 GET 방식의 /test API를 구현하였다.

-> 이미지 전송 전에 서버가 살아있는지 먼저 확인하기 위한 테스트용 API이다.

= 클라이언트와 서버 간 기본 간 연결 상태 확인

(2) 이미지 업로드 API 구현

클라이언트가 이미지를 서버로 전송할 수 있도록 POST 방식의 /upload API를 구현하였다.

FastAPI의 UploadFile을 사용하였다.

= 실제 데이터를 서버로 전달하는 역할

(3) 이미지 저장 기능 구현

업로드된 이미지 파일은 서버의 images 폴더에 저장되도록 하였다. 파일명은 timestamp를 이용하여 생성하였으며, 이를 통해 파일명이 겹치지 않도록 하였다.

-> timestamp: 파일 중복 방지, 시간 기준으로 파일을 관리하기 위해

(4) JSON 응답 구조 정의

서버에서 처리 결과를 클라이언트에 전달하기 위해 JSON 형식을 사용하였다.

응답 데이터는 status와 filename으로 구성하였다.

4. 코드

(1) 서버 코드

```
from fastapi import FastAPI, UploadFile, File
from datetime import datetime
import os

app = FastAPI()

UPLOAD_DIR = "images"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.get("/test")
def test():
    return {"status": "ok"}

@app.post("/upload")
async def upload(file: UploadFile = File(...)):
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"{timestamp}.jpg"
    file_path = os.path.join(UPLOAD_DIR, filename)

    contents = await file.read()
    with open(file_path, "wb") as f:
        f.write(contents)

    return {"status": "success", "filename": filename}
```

```

main.py > ...
1  from fastapi import FastAPI, UploadFile, File
2  from datetime import datetime
3  import os
4
5  app = FastAPI()
6
7  UPLOAD_DIR = "images"
8  os.makedirs(UPLOAD_DIR, exist_ok=True)
9
10 @app.get("/test")
11 def test():
12     return {"status": "ok"}
13
14 @app.post("/upload")
15 async def upload(file: UploadFile = File(...)):
16     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
17     filename = f"{timestamp}.jpg"
18     file_path = os.path.join(UPLOAD_DIR, filename)
19
20     contents = await file.read()
21     with open(file_path, "wb") as f:
22         f.write(contents)
23
24     return {"status": "success", "filename": filename}
25

```

```
PS C:\Users\ekdnj> pip install fastapi uvicorn requests python-multipart
```

라이브러리 설치

1) fastapi -> 이미지 업로드용 API 서버

-이유: 라즈베리파이에서 전송한 이미지를 pc에서 수신하고 YOLO 및 얼굴 인식 처리로 연결하기 위한 api 구조가 필요했기 때문

-장점: 파일 업로드 api 구현이 간단하고, 자동 문서화 기능이 있어서 개발과 테스트 하는데 용이하다.

그리고 python 기반으로 yolo, opencv, 얼굴 인식 모듈과 연동하기 쉽고, DB 저장 및 이벤트 조회하는 것에서 api를 단계적으로 늘려가기에 좋으므로(예: /upload, /events 등) 확장성이 좋다.

2) uvicorn -> 서버 실행 엔진(ASGI 서버 프로그램)

※ASGI(Asynchronous Server Gateway Interface): 웹 애플리케이션 (fastapi)와 실제 서버 사이를 연결해주는 표준 규칙, 동기 및 비동기 처리까지 하는 것으로 동시에 요청이 들어오면 효율적으로 처리하기 좋다.

->fastapi는 코드만 작성한다고 해서 자동으로 네트워크 요청을 받는 것이 아니므로(8000번 port 열어서, 브라우저 요청 받는 것 등) 실제로 요청을 받아주는 서버 프로그램이 필요하다. =uvicorn

3) requests -> HTTP 요청 보내기

4) python-multipart -> 파일 업로드 처리를 위해 필요한 패키지

```
PS C:\Users\ekdnj\Desktop\capstone> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\ekdnj\\Desktop\\capstone']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [18424] using StatReload
INFO: Started server process [23256]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

서버 실행(서버가 진짜 켜졌는지 확인하는 절차)

코드

```
@app.get("/test")
```

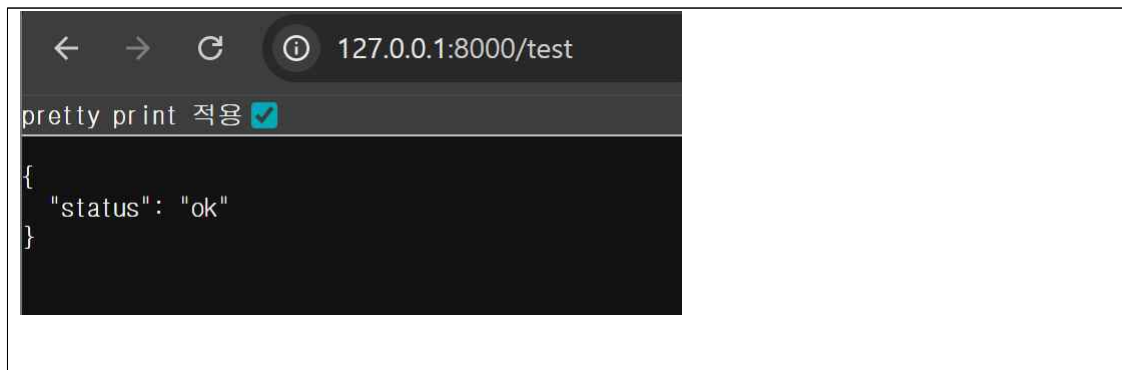
```
def test():
```

```
    return {"status": "ok"}
```

main.py 안의 서버를 실제로 실행-> <http://127.0.0.1:8000> 위의 실행 화면에서 볼 수 있듯이 서버 실행 완료

-127.0.0.1: 내 컴퓨터 자기 자신을 가리키는 주소(local)

-8000: 포트번호(서버가 8000번 포트에서 대기 중=고정)



(2) 클라이언트 코드

```
import requests

url = "http://192.168.0.113:8000/upload"

files = {"file": open("test.jpg", "rb")}

response = requests.post(url, files=files)

print(response.json())
```

```
PS C:\Users\ekdnj\Desktop\capstone> python test_client.py
{'status': 'success', 'filename': '20260402_125318.jpg'}
```

이미지 전송 실행 및 JSON 결과 확인

5. 실행결과

(1) JSON 응답

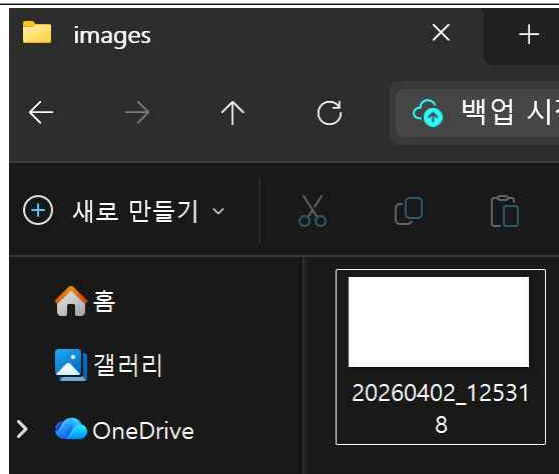
```
{
  'status': 'success', #(업로드 성공 여부 확인)
  'filename': '20260402_125318.jpg' #(저장된 결과 파일 추적)
```

```
}
```

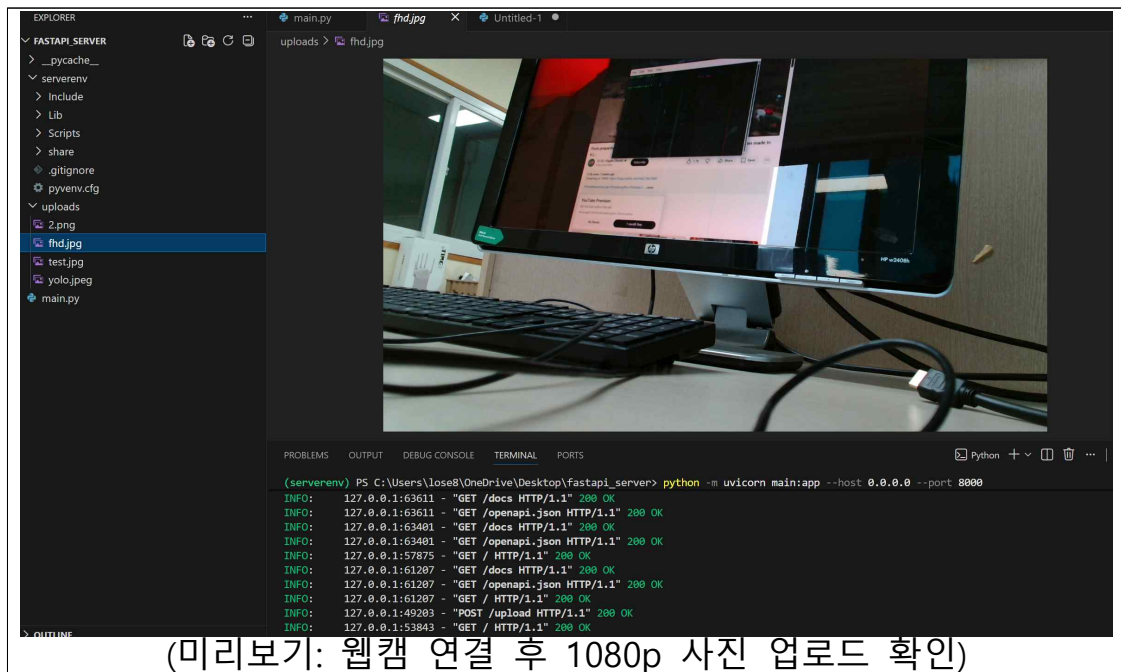
```
PS C:\Users\ekdnj\Desktop\capstone> python test_client.py  
{'status': 'success', 'filename': '20260402_125318.jpg'}
```

fastapi는 딕셔너리를 받아서 브라우저에 보낼 때 자동으로 json 형식으로 변환함

(2) 이미지 저장



이미지가 서버에 정상적으로 저장되는 것을 확인하였다.



(3) 서버 로그

```
PS C:\Users\ekdnj\Desktop\capstone> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\ekdnj\\Desktop\\capstone']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [19516] using StatReload
INFO: Started server process [7184]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:53750 - "POST /upload HTTP/1.1" 200 OK
```

서버가 정상적으로 요청을 처리하였다.

test_client.py(이후 라즈베리파이) 실행하니

"POST /upload HTTP/1.1" 200 OK

-POST방식으로 요청을 보냄(이미지 파일 업로드)

- 요청이 들어온 주소가 /upload API라는 뜻으로 이미지 업로드

기능 호출

-HTTP/1.1은 클라이언트와 서버가 HTTP/1.1 규칙으로 통신.(버전)

-200 OK는 서버가 요청을 정상적으로 처리

위와 같이 이미지 업로드 요청이 들어왔고 요청이 수행됨을 확인
(200 OK)

8000-> 서버 포트(고정)

53750-> 클라이언트 임시 포트(요청할 때마다 바뀜)

4. 결과

이미지 업로드 및 저장 기반의 통신 구조(클라이언트와 서버 간 통신 구조)가 정상 동작함을 확인하였으며, 이후 YOLO 객체 탐지 및 DB 저장 기능을 추가할 수 있는 기반을 확보하였다.

5. 이후 해야할 것(4차)

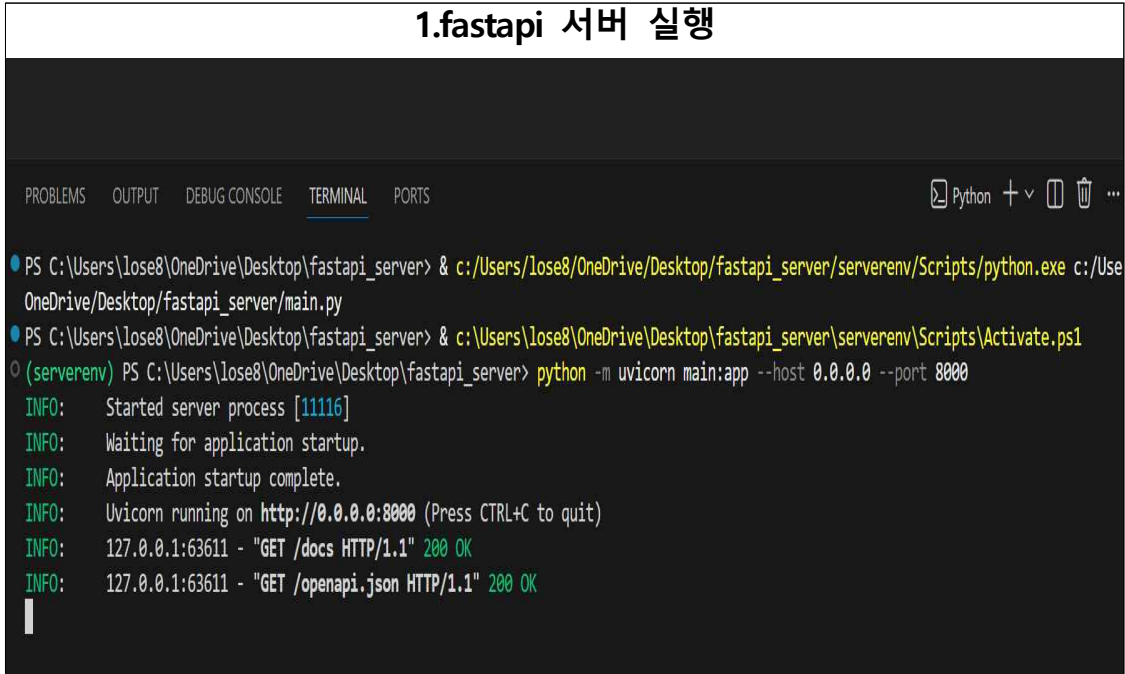
웹캠 -> 이미지 저장 -> 서버 전송

항목	3차	4차
클라이언트	Raspberry Pi	Raspberry Pi
이미지	test.jpg	카메라 캡처
서버	클라이언트와 같은 PC	PC 또는 서버

개인 수행 활동 모음(자료조사 등)

1. 처리, 서버

1.fastapi 서버 실행



```
PS C:\Users\lose8\OneDrive\Desktop\fastapi_server> & c:/Users/lose8/OneDrive/Desktop/fastapi_server/serverenv/Scripts/python.exe c:/Users/lose8/OneDrive/Desktop/fastapi_server/main.py
PS C:\Users\lose8\OneDrive\Desktop\fastapi_server> & c:/Users/lose8/OneDrive/Desktop/fastapi_server/serverenv/Scripts/Activate.ps1
(serverenv) PS C:\Users\lose8\OneDrive\Desktop\fastapi_server> python -m uvicorn main:app --host 0.0.0.0 --port 8000
INFO: Started server process [11116]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:63611 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:63611 - "GET /openapi.json HTTP/1.1" 200 OK
```

1. (serverenv)로 가상환경을 미리 활성화하여 가상환경 안에서만 패키지나 python, 서버 등을 동작하도록 하였습니다.

2. `python -m uvicorn main:app --host 0.0.0.0 --port 8000`

- ◇`python -m uvicorn` = uvicorn 서버를 실행
- ◇`main: app = main.py` 파일 안의 app이라는 객체를 실행(`app = FASTAPI( )`)
- ◇`--host 0.0.0.0` : 같은 네트워크의 다른 장치에서도 접속 가능하게 연결
- ◇`--port 8000` : 8000 번 포트에서 요청을 받겠다.

= 기존 코드 `uvicorn main:app`만 써도
pc(`http://127.0.0.1:8000/docs`)에서만 접속 가능하지만,
`--host 0.0.0.0 --port 8000`를 포함하여 같은 네트워크의 라즈베리 파이(`http://192.168.0.113:8000/docs`)에서도 접속 가능하게 할 수 있다.

3. started server process [11116]

-[11116]: 현재 실행된 서버 프로세스 번호 -> 서버 켜짐

4. 127.0.0.1:63611 - "GET /docs HTTP/1.1" 200 OK

-브라우저에서 fastapi 문서 페이지를 성공적으로 열었다.

5. 127.0.0.1:63611 - "GET /openapi.json HTTP/1.1" 200 OK

-/docs가 api 구조 정보를 담은 openapi.json 파일을 읽어서 화면을 만든다.

※ openapi.json = fastapi가 api 코드들을 바탕으로 자동 생성해서 /docs 페이지에 제공하는 api 구조를 가진 설명 데이터

【헛갈릴 수 있는 지식】

openapi = api 구조 설명하는 표준 규격

open api = 누구나 사용할 수 있도록 외부로 공개된 API 서비스

2. api 문서 페이지(/docs)

(1)GET

FastAPI 0.1.0 OAS 3.1
[/openapi.json](#)

default

GET / Read Root

Parameters

No parameters

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value

Schema

"string"

fastapi가 자동으로 제공하는 swagger UI 기반 api 문서 화면이다.

swagger ui = api 문서를 시각적으로 문서화하고 브라우저에서 api를 직접 테스트할 수 있도록 지원하는 도구

★fastapi -> openapi.json 자동 생성 -> swagger UI로 화면 표시

fastapi , OAS 3.1

- 서버는 fastapi기반이고 문서 형식을 openapi specification 3.1

기준으로 만들어졌다.

default

- 각종 api가 묶여 있는 곳으로 이후 프로젝트로 진행하면서 /upload, /detect , /events, /person 등과 같은 api를 추가

GET

-GET 방식 API 로 /docs 페이지를 열었을 때 실행되는 기능으로 서버가 정상적으로 켜져있는지 확인하는 용도로 만들었다.

클라이언트가 /에 접속하면 서버가 "message": "running"이라는 메시지를 돌려줌.

(1) parameters

요청할 때 필요한 입력값을 보여준다.

- 이 api는 별도의 입력값 없이 그냥 실행 가능하다.

(2) response

200 succesful response와 같은 서버가 정상 처리 했는지 보여준다.
응답 형식을 json 데이터 형식으로 설정하였음.

『try it out 실행 화면』

GET / Read Root

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/
```

Server response

Code

Details

200

Response body

```
{
  "message": "FastAPI is running"
}
```

Download

Response headers

```
content-length: 32
content-type: application/json
date: Mon, 06 Apr 2026 11:28:20 GMT
server: uvicorn
```

execute를 누르면 브라우저가 서버에 get요청을 보냄.

curl은 브라우저에서 한 테스트를 터미널 명령어로도 재현할 수 있게 보여주는 기능이다.

response body에서는 서버가 실제로 돌려준 데이터가 나온다.

```
{
  "message" : "fastapi is running"
}
```

Json 형식으로 돌려줌(서버 살아있다는 확인용 메시지)

(2)POST

POST /upload Upload Image ^

Parameters Try it out

No parameters

Request body required multipart/form-data

file * required
string

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type application/json	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": { "loc": ["string",], "msg": "string", "type": "string", "input": "string", "ctx": {} } }</pre>	No links

Schemas ^

Body_upload_image_upload_post > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

POST /upload

-POST 방식 API로 이미지 파일을 서버로 업로드하는 기능

(1) Request body (required)

이 api를 실행하려면 반드시 요청하는 파일 데이터를 같이 보내야 함.

(2) multipart/form -data

파일 업로드할 때 사용하는 전송 형식으로

/upload api는 json 형태로 받는 것이 아니라 파일 업로드용 형식으로 받도록 설계(파일과 텍스트를 같이 보낼 수 있음)

(3) response

'200 succesful response', '422 validation error' 와 같이 서버가 정상 업로드 성공했는지 보여주거나 입력값이 api 규칙에 맞지 않을 때 나는 오류(예: 파일을 안 넣거나, multipart/form -data 형식이 아닐 때)를 보여준다.

schemas

이 api가 사용하는 데이터 구조 목록

어떤 요청 본문(body)를 받는지 설명하는 구조,
입력 및 검증 시 나오는 오류 구조가 있다.

『try it out 실행 화면』

Request body required

multipart/form-data

file * required

파일 선택 2.png

string

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/upload' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@2.png;type=image/png'
```

Request URL

```
http://127.0.0.1:8000/upload
```

Server response

Code

Details

200

Response body

```
{
  "message": "upload success",
  "filename": "2.png"
}
```

Download

Response headers

```
content-length: 47
content-type: application/json
date: Mon, 06 Apr 2026 11:29:53 GMT
server: unicorn
```

파일을 꼭 넣어야 api가 실행됨.

curl에서 multipart/form -data 형식으로 이미지(png)를 파일로 첨부해서 보낸 것을 알 수 있다.

2. 전송, 통신

3. DB, 데이터

3-1

데이터흐
름
구조 설계

JSON(JavaScript Object Notation)?
JSON은 데이터를 저장하고 공유하기 위한 형식입니다.

```
{
  "name": "Ogre",
  "size": 90,
  "power": 86,
  "intelligence": 12,
  "magic": 0
}
```

JSON 객체는 중괄호 안에 키-값 쌍이 나열된 목록입니다. [2]

JSON 사용이유?

FastAPI의 기능 중 OpenAPI 자체가 JSON Schema를 기반으로 하기 때문입니다. [3]

1. JSON 응답 구조 설계

- 서버 처리 결과를 클라이언트에 전달하기 위해 JSON 형식을 사용했다. 응답 데이터는 최소한의 정보로 구성되어 결과와 파일명을 포함하도록 했다.

코드

```
return {"status": "success", "filename": filename}
```

결과

```
{
  "status": "success", #status->성공/실패
  "filename": "2026-04-07_101530.jpg"
}
```

2. timestamp 포맷 통일

- 시간데이터의 일관성 유지를 위해 "YYYY-MM-DD HH:MM:SS" 형식으로 통일했다.

코드

```
from datetime import datetime
timestamp =
datetime.now().strftime("%Y%m%d_%H%M%S")
```

3. 이미지 저장 경로 규칙 정의

- "images/파일명"

	• 폴더-> images • 파일명-> timestamp 기반 파일명을 사용하여 파일 중복을 방지하도록 했다.
	코드 <pre>timestamp = datetime.now().strftime("%Y%m%d_%H%M%S") filename = f"{timestamp}.jpg" file_path = os.path.join(UPLOAD_DIR, filename)</pre> 결과 예시 images/20260407_101530.jpg
	4. DB에 저장할 데이터 구조 확정 • 시스템에서 필요한 데이터를 기준으로 DB 구조를 설계했다. • 이미지 경로, timestamp, 객체 탐지 여부(detected), 이벤트 유형(event_type)에서 이미지 경로, timestamp, 전체 수신 여부, 탐지된 사람 수를 저장하도록 변경했다.
	코드 <pre>CREATE TABLE logs (id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp TEXT, image_path TEXT, all_received INTEGER, detected_persons INTEGER);</pre>
	5. 이후 insert 코드 작성 준비 • DB에 데이터를 저장하기 위해 INSERT 쿼리를 작성하고, Python에서 변수 형태로 데이터를 저장하도록 구현했다.
	코드 <pre>cursor.execute("INSERT INTO logs (timestamp, image_path, all_received, detected_persons) VALUES (?, ?, ?, ?)", (timestamp, file_path, 1, detected_persons)</pre>

	) 결과 이미지 업로드 시 자동으로 DB에 기록
3-2 API 데이터 형식 정의	API(Application Programming Interface)란? 정의 및 프로토콜 집합을 사용하여 두 소프트웨어 구성 요소가 서로 통신할 수 있게 하는 메커니즘입니다. 요청을 보내는 애플리케이션을 클라이언트라고 하고 응답을 보내는 애플리케이션을 서버라고 합니다. [1] 클라이언트-라즈베리파이 서버-PC API 사용이유? 개발 및 관리의 효율성 <형식 정의> 라즈베리파이: 이미지 전송-> multipart/form-data ↓ 서버: 이미지 저장 + 처리 + JSON 응답 반환 POST /upload (이미지업로드) 동작과정 (1) 클라이언트(라즈베리파이)에서 이미지 전송 (2) 서버에서 이미지 파일 저장 (3) timestamp 기반 파일명 생성 (4) DB에 저장 (5) JSON 형태로 결과 반환 GET /test(서버 상태 확인) 동작과정 (1) 클라이언트에서 요청 전송 (2) 서버에서 정상 동작 여부 확인 (3) 상태 값을 JSON으로 반환
	클라이언트

	<pre> files = {"file": open("test.jpg", "rb")} response = requests.post(url, files=files) print(response.json()) </pre> 서버 <pre> async def upload(file: UploadFile = File(...)): </pre>
--	--

출처

[\[1\] 애플리케이션 프로그래밍 인터페이스\(API\)란 무엇인가요?](#)

[\[2\] What is JSON?](#)

[\[3\] FastAPI](#)

[\[4\] 동기 비동기 방식 설명 및 환경 구축](#)

[\[5\] 파이썬으로 웹 api 만들기](#)

[\[6\] openapi, swagger](#)